

BENNETT UNIVERSITY

School of Computer Science Engineering and Technology
Greater Noida, Uttar Pradesh

SCSE3040 Machine Learning Operations

Lecture 01 Handout

and Laboratory Companion

From a notebook that runs once, to a service that runs every day.

Programme	B.Tech Computer Science and Engineering
Semester	Semester V, Odd Semester, Session 2026 to 27
Credits	3 (L, T, P, E structure: 2, 0, 2, 0)
Contact hours	30 lecture and 30 laboratory, 60 in total
Assessment	Continuous Assessment 60, End Semester Examination 40
Course coordinator	Dr. Gaurav Tripathi
Lecture faculty	Mr. Dhananjay Kumar, dhananjay.kumar@bennett.edu.in
Lecture slots	Wednesday and Friday, 15:55 to 16:55, venue P-LH-101
Batches	B74 to B93
Consultation hours	_____ to _____

Issue 1.0, 18 August 2026

Issued under the approved Teaching Learning and Assessment Plan for SCSE3040.

All external links in this handout were checked on 18 August 2026.

How to use this handout

This single document carries everything you need to begin the course. Keep it for the whole semester. It has four parts.

Part I	The course: what it is, what you will be able to do at the end, the syllabus, the running project and the fixed tool list.
Part II	Assessment and policy: every component, every rubric, every rule, reproduced in full so that nothing about your marks is a surprise.
Part III	The laboratory companion: the complete environment setup, one command at a time, and Practical 01 in detail.
Part IV	Resources: the reading list, the official manuals, and answers to the questions students actually ask.

Read this before the first laboratory session

Part III, Section 15, is the setup guide. Work through it before your first practical if you possibly can. A student who arrives with a working environment spends the session learning MLOps. A student who arrives with nothing spends the session installing Python.

Contents

How to use this handout	1
I The Course	4
1 Course identity	4
2 What this course is about	4
2.1 The gap, in one story	4
2.2 A notebook, and a production service	5
2.3 So what is MLOps	5
2.4 The quiet failure	5
2.5 The lifecycle is a loop, not a line	5
2.6 What MLOps adds to DevOps	6
2.7 Four beliefs to leave behind today	7
3 Course objectives	7
4 Course outcomes	7
4.1 Mapping to programme outcomes and programme specific outcomes	8
5 Syllabus	8
6 The running project	9

7 The tool list, and why it is closed	10
II Assessment and Policy	11
8 Assessment components	11
9 Continuous Lab Evaluation rubric, 4 marks per session	11
10 Assignment rubric, 10 marks	12
11 Self Learning Assignment rubric, 10 marks	12
12 Attainment levels	13
13 Course policies	13
13.1 Attendance, punctuality and participation	13
13.2 Academic integrity and the use of AI tools	14
13.3 Pre reading and the weekly rhythm	14
13.4 Discipline	14
13.5 The laboratory environment	14
13.6 Remedial support	14
14 Rhythm of the semester	14
III Laboratory Companion	15
15 Setting up your machine	15
15.1 Step 1: install Python	15
15.2 Step 2: install Git and set your identity	16
15.3 Step 3: create your GitHub account and prepare authentication	16
15.4 Step 4: get the course repository	16
15.5 Step 5: create the course virtual environment	17
15.6 Step 6: install the course libraries	17
15.7 Step 7: register the notebook kernel	17
15.8 Step 8: launch and verify	18
15.9 Working behind the campus proxy, or with no network	18
15.10 Docker Desktop, and why you must not install it yet	18
16 Troubleshooting	18
17 Practical 01: Your MLOps Workbench	20
17.1 Aim	20

17.2 Why this practical exists	20
17.3 The walkthrough	20
17.4 Your tasks	21
17.5 The self check	22
17.6 What to submit	22
17.7 How this practical is marked	23
17.8 Questions you must be ready to answer	23
18 The laboratory roadmap	23
IV Resources	25
19 Reading list	25
19.1 Prescribed textbooks	25
19.2 Reference books	25
19.3 Free and openly available	25
20 Official documentation, the real textbook of this course	26
21 Frequently asked questions	27
Appendices	29
A Before the next session: your checklist	29
B Acknowledgement of the course handout	29
C Faculty annex	29
C.1 Before the session	29
C.2 Timing for a 120 minute block	30
C.3 Say these things out loud	30
C.4 Where students get stuck, and what to say	30
C.5 Records	31

PART I

The Course

1 Course identity

Course code	SCSE3040
Course title	Machine Learning Operations
Programme and semester	B.Tech Computer Science and Engineering, Semester V, Odd Semester
Credits	3
L, T, P, E	2, 0, 2, 0
Contact hours	2 lecture and 2 laboratory hours per week, 60 in total
Assessment split	Continuous Assessment 60, End Semester Examination 40
End semester paper	40 marks, pen and paper
Aggregate pass mark	40 percent
Prerequisite	None formally
Course coordinator	Dr. Gaurav Tripathi
Your lecture faculty	Mr. Dhananjay Kumar
Contact	dhananjay.kumar@bennett.edu.in
Chamber consultation	_____ to _____

There is no formal prerequisite for this course. Prior exposure to Python and to basic machine learning helps, and for students without it a compressed machine learning bridge and learner support sessions are built into the session plan. Nobody is expected to arrive already knowing MLOps.

2 What this course is about

You can already train a model. You cannot yet hand it to a stranger and have it keep working. That gap, between a model that runs once for you and a model that runs every day for everyone, is exactly what this course closes.

2.1 The gap, in one story

Four students build a delivery time predictor. The accuracy is good. The demonstration gets applause. A restaurant chain asks whether they can use it from tomorrow, and within a week three things surface.

- It runs on one laptop, and nobody else can call it: not a teammate, not the chain's application.
- It needs a file sitting in a Downloads folder, because there is no endpoint another program can talk to.
- Three months later, order patterns change, the predictions quietly get worse, and nobody notices, because nothing is watching.

Cooking one dish at home is not the same as running a kitchen for five hundred plates a night. Same recipe, completely different demands: consistency, speed, staffing, and a plan for when something goes wrong.

2.2 A notebook, and a production service

In a notebook	In production
Runs when you press Shift and Enter	Runs on its own, day and night
One user, and that user is you	Many users, at once
Breaks silently	Must be watched, and must alert someone
Data sits in a folder	Data keeps arriving

2.3 So what is MLOps

MLOps is the set of practices that takes a model out of a notebook and keeps it running, reliably, for real users. Three properties define it.

The three properties

Repeatable. Anyone on the team can rebuild the same model from the same data and the same code.

Reachable. Other software can call it: an application, a website, another service, not just you in a notebook.

Watched. When it gets worse, somebody is told automatically, before customers notice.

None of that is about accuracy. A brilliant model that nobody can reach, trust or reproduce is not a production system. These demands sit on top of accuracy, not instead of it, and they are what this whole course is about.

What production actually demands is a short and unforgiving list. It runs without you being awake. It answers in milliseconds, not minutes. It gives the same answer today and next Tuesday for the same input. It can be rolled back the moment it goes wrong. And it leaves a trail: who trained it, on what data, and when.

2.4 The quiet failure

A web server tells you when it crashes. The alert fires, and somebody's phone rings at two in the morning. Loud and immediate.

A model does not tell you when it starts being wrong. It keeps answering, confidently, while the uptime dashboard stays green the entire time. Day one it is accurate, day thirty slightly off, day sixty often wrong, day ninety wrong more often than not, and the server was never once down.

That asymmetry is why monitoring in this course means prediction quality and input drift, not merely whether the service is up.

2.5 The lifecycle is a loop, not a line

Most students picture machine learning as a straight line: data, train, done. In practice it is a circuit of six stages.

Stage	What happens
1. Collect data	Orders, distances, delivery times. Messy, incomplete, and arriving all the time. Never a clean finished table.
2. Build features	Turn raw rows into numbers a model can use: distance in kilometres, whether it is raining, how busy the kitchen is.
3. Train	Try settings, get a model, try again. The smallest box on the diagram, even though it feels like the main event.
4. Evaluate	Score it on data it has never seen. Never on the rows it trained on.
5. Deploy	Wrap the model so other software can call it, then put it somewhere that stays switched on.
6. Monitor	Watch the answers, not just the server. A healthy server can still be quietly giving wrong predictions.

The arrow that matters runs from monitor back to collect data. What you learn while watching a live model becomes next month's training data. A straight line picture of this work quietly assumes the world holds still after deployment. It never does, which is why finished is the wrong word for stage three.

2.6 What MLOps adds to DevOps

Software teams solved a version of this problem twenty years ago. Developers wrote code and threw it over the wall, operations ran it and took the two in the morning call, and releases happened twice a year on a Friday night. DevOps was the fix: one team, one shared pipeline, automation instead of a wall.

MLOps borrows four DevOps habits unchanged. Version everything. Automate the test and release path. Make small frequent changes rather than large rare ones. And if a manual step hurts, script it away first.

Then it adds what only appears once a model is involved.

	DevOps	MLOps
What you version	Code	Code, data and model
What tests check	Does the code work	Is the model still accurate
Why it breaks	Someone changed the code	The world changed
What you roll back to	The previous build	The previous model version

Code rots slowly, and only when somebody touches it. Models rot on their own. Change any one of code, data or model and the behaviour changes, so all three need versions and all three need testing.

2.7 Four beliefs to leave behind today

The belief	Why it fails
My accuracy is 95 percent, so the work is done	Accuracy is the start, not the finish
I will deploy it at the end	Deployment shapes how you build from day one
MLOps is just DevOps with a new name	Data and models change on their own; code does not
This only matters for large companies	A two person startup feels it fastest, because nobody there has time to babysit a notebook

3 Course objectives

1. To introduce the foundational principles, history and lifecycle of Machine Learning Operations, and the transition of a machine learning model from a standalone notebook to a reliable production service.
2. To establish sound data foundations for production machine learning, including data collection, schema validation, the theory of data cascades, and version control of code and datasets using Git and DVC.
3. To develop software engineering competence for machine learning applications, covering object oriented project structure, YAML configuration, testing, experiment tracking and model management with MLflow, and model serving through REST APIs.
4. To build practical capability in reproducible environments and containerization using Docker and multi service orchestration with Docker Compose, with a conceptual understanding of Kubernetes based production orchestration.
5. To provide experiential learning of automation and reliability in production machine learning through CI/CD pipelines with GitHub Actions, workflow orchestration with Apache Airflow, drift monitoring with Evidently AI unified in MLflow, and responsible deployment practices.

4 Course outcomes

At the end of this course you will be able to do the following.

CO	Statement	Bloom level
CO1	Explain and articulate the fundamental principles of MLOps, the production machine learning lifecycle, data foundations and data cascades, and the version control of code and datasets.	Understanding, BTL 2
CO2	Execute software engineering workflows for machine learning applications, object oriented paradigms, YAML configuration and testing, experiment tracking and model management with MLflow, and model serving and deployment on cloud platforms.	Applying, BTL 3
CO3	Execute containerization workflows and configure reproducible environments, Docker images and container management, multi service orchestration with Docker Compose, and a conceptual understanding of Kubernetes based production orchestration.	Applying, BTL 3

CO	Statement	Bloom level
CO4	Construct microservices and REST API endpoints for machine learning deployment, and examine CI/CD pipeline configurations using GitHub Actions.	Analyzing, BTL 4
CO5	Assess end to end automated machine learning pipelines from data to production readiness, using workflow automation with Apache Airflow and model monitoring with Evidently AI unified in MLflow, and appraise deployed models for sustainable bias and ethical impact.	Evaluating, BTL 5

4.1 Mapping to programme outcomes and programme specific outcomes

Scale: 3 strongly mapped, 2 moderately mapped, 1 weakly mapped, a dash for not mapped.

CO	PO1	PO2	PO3	PO4	PO5	PO6
CO1	3	3	1	1	–	–
CO2	3	3	3	2	3	1
CO3	2	3	3	2	3	1
CO4	2	3	3	2	3	2
CO5	3	3	2	3	3	3
Contribution	2.60	3.00	2.40	2.00	3.00	1.75

CO	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	–	–	–	1	–	3	3	1	2
CO2	–	1	1	2	2	3	3	3	2
CO3	–	1	2	2	2	3	3	3	2
CO4	–	2	2	2	2	3	3	3	2
CO5	1	3	1	3	2	3	2	3	3
Contribution	0.20	1.75	1.50	2.00	2.00	3.00	2.80	2.60	2.20

Three of these deserve a sentence of explanation, because they shape how the course is taught. **PO5, modern tool usage**, is strongly addressed throughout, since MLOps is fundamentally a modern tools discipline and every tool is applied with its limitations discussed. **PO8, ethics**, is carried by the responsible MLOps content and reinforced by the disclosure based academic integrity policy in Section 13.2. **PO12, lifelong learning**, is why this course trains you to read official documentation rather than to memorise a tool version that will be obsolete before you graduate.

5 Syllabus

Module Content		Hours
I	Introduction to MLOps, data foundations and data cascades. History and evolution; core principles and the production machine learning lifecycle; DevOps to MLOps; bridging data science, DevOps and engineering roles; a machine learning bridge covering model families, train and test methodology, evaluation metrics and model artifacts; AutoML by demonstration; data cascades in high stakes AI, physical world brittleness, domain drift, goodness of fit against goodness of data; data collection, ingestion and schema validation; version control philosophy with Git; data versioning with DVC.	8
II	Software engineering, experiment tracking and model serving. Software development paradigms for machine learning applications, object oriented design, project structuring and refactoring; configuration management with YAML; software testing with pytest; experiment tracking of parameters, metrics and artifacts; MLflow with a local tracking server backed by SQLite; the MLflow Model Registry and model lifecycle management; serving fundamentals, REST architecture, JSON payloads and HTTP methods; native serving with MLflow; building model APIs with FastAPI; cloud deployment overview of AWS SageMaker and GCP Vertex AI, and practical free deployment paths.	8
III	Containerization and multi service orchestration. Reproducible environments and the works on my machine problem; containers against virtual machines; images, layers and the Dockerfile; containerizing the machine learning service; Docker networking and volumes; multi service orchestration with Docker Compose; composing the tracking server and the serving API; deployment pipeline patterns for data, model and deploy pipelines; logging fundamentals for containerized services; Kubernetes as a conceptual overview of production orchestration, theory only.	7
IV	CI/CD automation, monitoring and responsible MLOps. CI/CD for machine learning, automated testing, building and deployment; automated tests for data schemas and model input and output; YAML pipeline structures with GitHub Actions; a brief perspective on Jenkins, ArgoCD and GitLab CI; workflow automation with Apache Airflow, DAGs, operators and scheduling; an introduction to streaming with Kafka; model monitoring, concept drift against data drift; drift detection with Evidently AI; unified monitoring by logging drift metrics to MLflow; Grafana dashboards by demonstration; responsible MLOps, bias, ethics and production readiness audits.	7

6 The running project

There is one project, and it runs all semester: **a delivery time predictor**. Every laboratory session moves it one step forward, and nothing is thrown away.

The semester arc

environment → versioned data → tracked experiments → a served model → a deployed service
→ a container → a composed stack → an automated pipeline → monitoring

Laboratories 1 to 10 build the fundamentals, one gentle step per week. Laboratories 11 to 13 add the advanced layer: CI/CD automation, workflow orchestration, and drift monitoring unified in MLflow. Laboratories 14 and 15 are buffer sessions with no fixed topic, held in reserve for catching up, revision and project work.

All laboratory work is CPU only and designed for standard laboratory systems with 8 GB of RAM, with no GPU dependency anywhere in the course.

7 The tool list, and why it is closed

One tool per job, chosen once, used all semester. Substitutions are not accepted, because the oral assessments, the codathons and the end semester paper are all set against this list.

Job	Tool	Official manual
Code versioning	Git and GitHub	https://git-scm.com/book/en/v2
Data versioning	DVC	https://doc.dvc.org/start
Modelling	scikit-learn	https://scikit-learn.org/stable/user_guide.html
Testing	pytest	https://docs.pytest.org/en/stable/getting-started.html
Experiment tracking and registry	MLflow	https://mlflow.org/docs/latest/ml/getting-started/
Model API	FastAPI	https://fastapi.tiangolo.com
Containers	Docker and Docker Compose	https://docs.docker.com/get-started/
Continuous integration and delivery	GitHub Actions	https://docs.github.com/en/actions/get-started/quickstart
Workflow orchestration	Apache Airflow	https://airflow.apache.org/docs/apache-airflow/stable/start.html
Drift monitoring	Evidently AI	https://docs.evidentlyai.com/quickstart_ml

Kubernetes is treated conceptually, for industry literacy. Kafka and Grafana appear as demonstrations only. You are not expected to run a cluster on a laboratory machine.

PART II

Assessment and Policy**8 Assessment components**

Because the structure carries a practical component, the Continuous Lab Evaluation is mandatorily part of Continuous Assessment. All graded assessments are conducted in pen and paper or live laboratory demonstration mode, as per SCSET examination guidelines. The Socio-Ed quizzes that appear in the session plan are formative and carry no marks.

Component	What it is	Weight
Continuous Lab Evaluation	Assessed across 15 laboratory sessions on the semester running project. Attendance, execution and oral assessment, all judged in the session. The best 10 scores count, at 4 marks each.	40 percent
Assignment	Applied work on the running project, operationalised through the two course codathons: Notebook to API, and Ship It, containerize and use CI/CD.	10 percent
Self Learning Assignment	Independent study of an approved emerging MLOps topic beyond the classroom track, with a submitted report and an oral assessment. Topics are released mid semester.	10 percent
End Semester Examination	Comprehensive pen and paper examination across the complete syllabus, with analytical, application oriented and evaluative questions across all course outcomes.	40 percent

Marks distributed across the outcomes: CO1 16, CO2 24, CO3 17, CO4 21, CO5 22, totalling 100.

The most important sentence in this handout

Forty percent of your total marks are earned live, in the laboratory, week by week. Those marks are awarded during the session against the rubric below. There is no route to them from home, and the LMS submission point is a record of your work rather than the place where marks are decided.

9 Continuous Lab Evaluation rubric, 4 marks per session

Component	Max	Descriptors and performance indicators
Attendance	1.0	Present for the complete laboratory session 1.0; late arrival or early departure 0.5; absent 0.0.

Component	Max	Descriptors and performance indicators
Execution	1.0	Correct and complete execution of the experiment with minimal guidance 1.0; partial execution or minor errors 0.5; unable to execute or major errors 0.0.
Oral assessment	2.0	Clear understanding of concepts, procedure and outcomes, answered confidently 2.0. For the CO5 laboratories, sessions 12 and 13, this means critically evaluating pipeline outputs, drift reports and bias metrics, and justifying recommended actions with evidence. Partial understanding, answers with prompts 1.0 to 1.5. Negligible understanding, incorrect or vague responses 0.0.

The best 10 laboratory performances are considered, irrespective of the total number of laboratories conducted: 10 evaluations at 4 marks gives the full 40, which is 40 percent of the total assessment.

Absence

If you are absent for a laboratory session, zero is awarded for all components of that session. Absence in two consecutive sessions triggers the SCSET parent communication process.

10 Assignment rubric, 10 marks

Component	Max	Levels
Timely submission	3.0	Submitted on or before the deadline, complete, original and strictly as per guidelines 3.0; submitted after the deadline but within the allowed grace period 2.0; very late, incomplete or not as per guidelines 1.0; not submitted 0.0.
Completeness	4.0	All questions and tasks fully attempted as per instructions, nothing missing 4.0; most attempted, minor parts missing 3.0; some attempted, several parts missing 2.0; very few attempted, largely incomplete 1.0; not attempted or completely irrelevant 0.0.
Oral assessment	3.0	Excellent understanding of concepts, procedures and outcomes, answers confidently 3.0; good conceptual clarity with minor gaps 2.0; basic understanding, partially correct, limited explanation 1.5; minimal understanding, unable to explain clearly 1.0; incorrect, vague or absent 0.0.

If the assignment is not submitted, no other rubric component is assessed and the student is awarded zero marks.

11 Self Learning Assignment rubric, 10 marks

Component	Max	Levels
Timely submission	3.0	As for the assignment rubric above.
Quality and depth of content	2.0	Comprehensive, well researched, showing critical thinking and clear understanding 2.0; good and mostly accurate, moderate depth 1.5; basic, partially correct 1.0; inadequate or largely incorrect 0.5; irrelevant, incorrect or not attempted 0.0.
Originality and self learning	2.0	Strong originality and independent learning, new ideas or approaches, effective use of multiple resources 2.0; some originality, moderate independent effort, appropriate use of references 1.5; limited originality, relying mostly on guided material 1.0; minimal independent effort, or indications of copying 0.5; entirely copied, or no evidence of self learning 0.0.
Oral assessment	3.0	Excellent understanding of the self learned topic, critically appraising trade offs, limitations and applicability, defending evaluative conclusions with evidence 3.0; good conceptual clarity with minor gaps 2.0; basic understanding, partially correct 1.5; minimal understanding 1.0; incorrect, vague or absent 0.0.

12 Attainment levels

Level	Criterion
Level 3	80 percent or more of students score above 60 percent marks
Level 2	70 percent or more and under 80 percent
Level 1	60 percent or more and under 70 percent
Below Level 1	Under 60 percent, needs improvement

Direct assessment comes from the Continuous Lab Evaluation, the assignment codathons, the Self Learning Assignment and the end semester examination, all mapped to the course, programme and programme specific outcomes. Indirect assessment comes from the course exit survey and student feedback collected during the university feedback window.

13 Course policies

13.1 Attendance, punctuality and participation

You are expected to be punctual and regular in attending classes and in submitting assignments and activities. Class participation is judged on punctuality, frequency and level of interaction, and contribution to the learning process.

13.2 Academic integrity and the use of AI tools

The rule, stated plainly

AI tools may be used as learning aids, provided that the use is disclosed in the README of your submission. Every student must be able to explain and modify their submitted work in the oral assessment. Undisclosed or unexplainable AI generated work is treated as plagiarism under university policy.

The second sentence is the one that decides marks. The oral assessment is where a submission you cannot explain becomes visible, and it is worth half of every laboratory session.

13.3 Pre reading and the weekly rhythm

The readings and references given in the session wise plan are to be read *before* coming to class, so that the discussion in the session is meaningful. Weekly material is released on the Socio-Ed platform by Monday at 08:00.

13.4 Discipline

Students are expected to be punctual and regular, to be decently dressed, and to be respectful and sensitive to everyone in their interactions.

13.5 The laboratory environment

All student laboratory work is CPU only and designed for standard laboratory systems with 8 GB of RAM, with no GPU dependency. Kafka and Grafana appear only as instructor demonstrations.

13.6 Remedial support

Non graded practice quizzes act as pace checks at the end of each module. If the median for a section falls below 60 percent, a remedial session is triggered for that section. Tell your faculty in week one if you are struggling, not in week eight.

14 Rhythm of the semester

When	What
Every Monday, 08:00	The week's material goes live on the Socio-Ed platform
Every week	One laboratory session on the running project, marked in the session
Twice in the semester	An industry expert talk: production machine learning platforms, and CI/CD and platform engineering for AI systems
End of each module	A consolidation activity: recap video, discussion forum and a non graded practice quiz
Mid semester	Self Learning Assignment topics released
Sessions 31 and 32	Codathon 1, Notebook to API
Sessions 46 and 47	Codathon 2, Ship It, containerize and use CI/CD
Sessions 57 to 60	Buffer laboratories, held in reserve

PART III

Laboratory Companion

15 Setting up your machine

This section takes you from a bare laptop to a verified working environment. Do it before the first practical if you can. It takes about fifteen minutes on a good connection.

Component	Version	Why the course needs it
Python	3.12 or newer	The language every practical is written in
Virtual environment	Built into Python	A private library folder for this course only, so installing something here cannot break another project. This is the entire subject of Practical 01
Git	Any current version	Records every version of your work, and is the backbone of Practicals 01 and 11
GitHub account	Free	Where your repository lives and where the pipeline in Practical 11 runs
JupyterLab	From the course requirements	Opens and runs the notebook files
Docker Desktop	Latest stable	Needed only from Practical 08. Do not install it now

Four checks before you touch a keyboard

1. You have administrator rights, or the laboratory image already carries Python and Git.
2. You have about 5 GB of free disk space.
3. You know whether the machine reaches the internet directly or through a campus proxy. If through a proxy, read Section 15.9 first.
4. Your course folder is *not* inside a folder that a cloud client is synchronising, such as OneDrive, Google Drive or Dropbox. Synchronising clients lock files while Git and pip are writing to them, and the resulting errors are confusing and hard to diagnose.

15.1 Step 1: install Python

Download the installer from the official source: <https://www.python.org/downloads/>

Windows. On the very first installer screen, tick **Add python.exe to PATH** before clicking Install. If you miss it, nothing else in this guide will work and you will have to reinstall.

macOS. Use the installer from the same page, or `brew install python` if you already have Homebrew.

Linux. Python is almost certainly present. Confirm with the check below, and install the `venv` package through your package manager if the environment step later complains.

Verify. On macOS and Linux use `python3` in place of `python`.

```
python --version
```


You should see Python 3.12.x or higher.

If a Microsoft Store window opens instead

Windows has handed you a stub rather than a real Python. Go to Settings, then Apps, then Advanced app settings, then App execution aliases, and switch *off* both `python.exe` and `python3.exe`. Then try again.

Official manual: <https://docs.python.org/3/using/index.html>

15.2 Step 2: install Git and set your identity

Download Git from <https://git-scm.com/download/win> on Windows, or run `brew install git` on macOS. Accept every default in the Windows installer. Verify with `git -version`.

Now tell Git who you are. Git stamps every commit with this name and address, so use ones you will not be embarrassed by in a viva.

```
git config --global user.name "Your Full Name"
git config --global user.email "your.id@bennett.edu.in"
git config --global init.defaultBranch main
```

On Windows, add this line as well. It stops the line ending warnings that otherwise appear on every commit.

```
git config --global core.autocrlf true
```

Official manual: <https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup>

15.3 Step 3: create your GitHub account and prepare authentication

Create a free account at <https://github.com> using an address you will keep after graduation.

GitHub will not accept your account password from the command line

When Git asks for a password while pushing, it wants a personal access token instead. Create one before the session so you are not stuck at the last step of Practical 01.

1. On GitHub, open Settings, then Developer settings, then Personal access tokens.
2. Generate a new token. Name it something you will recognise, and set an expiry at the end of the semester.
3. Grant it the `repo` scope, which is what command line access to your repositories requires.
4. Copy the token immediately. GitHub shows it once and never again. Paste it as the password when Git prompts you.

A token is a password and carries the same risks. Never paste one into a notebook, a screenshot or a shared document.

An alternative that avoids tokens entirely is Git Credential Manager, which ships with Git for Windows and handles the browser sign in for you. If it appears during your first push, use it.

Official manual: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

15.4 Step 4: get the course repository

The repository address for this section is: to be announced in the first session

```
git clone <the repository address your instructor gives you> MLOps
```

```
cd MLOps
```

Keep the folder path short and free of spaces. On Windows, a folder directly on the C drive works well.

15.5 Step 5: create the course virtual environment

From inside the course folder:

```
python -m venv .venv
```

On macOS and Linux use `python3` in place of `python`. This creates a folder named `.venv`. It is your private Python for this course, and you never edit anything inside it.

You do not have to activate it. Throughout this course we call the environment Python directly, which is the safest habit and behaves identically in every shell.

```
Windows:          .venv\Scripts\python
macOS and Linux:  .venv/bin/python
```

If you prefer to activate it and PowerShell refuses with a script execution error, run this once in that window and try again.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

Official manuals: <https://docs.python.org/3/library/venv.html> and <https://docs.python.org/3/tutorial/venv.html>

15.6 Step 6: install the course libraries

```
Windows
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\python -m pip install -r requirements-lock.txt
.venv\Scripts\python -m pip install jupyterlab nbformat ipykernel httpx

macOS and Linux
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements-lock.txt
.venv/bin/python -m pip install jupyterlab nbformat ipykernel httpx
```

This downloads roughly 400 MB, once. The lock file pins an exact version of every library so that every machine in the room ends up identical. That is the whole point of it, and Practical 01 explains why it matters.

Official manuals: <https://pip.pypa.io/en/stable/reference/requirements-file-format/> and https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html

15.7 Step 7: register the notebook kernel

```
Windows
.venv\Scripts\python -m ipykernel install --sys-prefix --name python3 --display-name "
    ↪ Python 3 (SCSE3040)"

macOS and Linux
.venv/bin/python -m ipykernel install --sys-prefix --name python3 --display-name "Python
    ↪ 3 (SCSE3040)"
```

This tells Jupyter which Python to run notebooks with. Skip it and your notebooks may silently pick a different Python, at which point half the imports fail for no visible reason.

15.8 Step 8: launch and verify

```
Windows
cd labs\P01-workbench
..\..\venv\Scripts\python -m jupyter lab P01.ipynb

macOS and Linux
cd labs/P01-workbench
../../venv/bin/python -m jupyter lab P01.ipynb
```

Your setup is correct when, and only when, both of these are true

1. Step 0 of the notebook prints All good.
2. Step 1 of the notebook prints Inside .venv : True

If the second line says False, Jupyter was started by the wrong Python. Close it and relaunch with the exact command above, including the leading path to the environment.

15.9 Working behind the campus proxy, or with no network

Package installation is the one step that depends on the network, and it is the step most likely to fail in a full laboratory.

If pip fails with an SSL or connection error, ask your instructor which of these applies.

```
pip install --proxy http://USER:PASSWORD@PROXYHOST:PORT -r requirements-lock.txt
```

Offline route, prepared by the instructor in advance

On any machine with a working connection, using the same operating system and Python version as the laboratory:

```
python -m pip download -r requirements-lock.txt -d wheels
```

Copy the wheels folder to the laboratory machines, then on each one:

```
..venv\Scripts\python -m pip install --no-index --find-links wheels -r requirements-
↪ lock.txt
```

This turns a forty minute network stall into a two minute local copy, and it is the single most effective piece of preparation for a first laboratory session.

15.10 Docker Desktop, and why you must not install it yet

Docker is needed only from Practical 08. It runs in the background, holds about 2 GB of memory, and will slow every other practical if you install it in week one.

When Practical 08 approaches, download from <https://www.docker.com/products/docker-desktop/>, install, restart when asked, and wait until Docker Desktop reports that the engine is running. Verify with `docker version`: you must see both a Client section and a Server section, because Client only means the engine has not started. On an 8 GB machine, open Settings, then Resources, and set memory to 2 GB, and close your browser before the session.

Official manuals: <https://docs.docker.com/get-started/> and <https://docs.docker.com/desktop/setup/install/windows-install/>

16 Troubleshooting

Symptom	Cause	Fix
python opens the Microsoft Store	Windows app execution alias	Settings, Apps, Advanced app settings, App execution aliases, turn both off
'python' is not recognized	Add to PATH was not ticked	Reinstall Python and tick the box on the first screen
PowerShell blocks the activate script	Execution policy	Bypass it for that window, or simply call the environment Python directly
ModuleNotFoundError for a library you installed	Wrong interpreter, not a missing library	Always call the environment Python, never a bare python
Inside .venv : False	Jupyter launched by the wrong Python	Relaunch with the full path shown in Step 8
Kernel Python 3 (SCSE3040) missing	Step 7 skipped	Run the ipykernel command, then restart Jupyter
pip fails with SSL or proxy errors	Campus network	Use the proxy or offline routes in Section 15.9
Support for password authentication was removed	GitHub password used instead of a token	Create a personal access token, Step 3
git commit says nothing to commit	The cell was run twice	Harmless. The first run already committed
Warnings about LF and CRLF	Windows line endings	Set the autocrlf option once. Harmless either way
Files locked, or permission denied	A cloud client is synchronising the folder	Move the course folder outside the synchronised area
Everything is very slow	8 GB of RAM with a browser open	Close the browser tabs. On this hardware the browser is usually the real problem

The five golden rules

1. Never run a bare `python`. Always call the environment Python.
2. Never commit a dataset or a model file to Git. That is what data versioning is for.
3. Never edit anything inside `.venv`.
4. Run notebook cells in order, from the top. A `NameError` almost always means a skipped cell.
5. Bring the exact error text to the laboratory, not a description of it from memory.

17 Practical 01: Your MLOps Workbench

Practical	P01, Your MLOps Workbench
Follows lectures	L01 and L02
Course outcome	CO1
Duration	120 minutes
Peak memory	About 250 MB
Extra software	None beyond the course environment

17.1 Aim

1. Find out which Python is actually running your code.
2. Write down the exact library versions your project needs.
3. Make a program that gives the same answer every single time.
4. Save your work in Git with a proper first commit.

17.2 Why this practical exists

Almost every problem in production machine learning starts with one sentence: *but it works on my machine*.

It works on your machine because your machine has one particular Python, with particular libraries, at particular versions, and because the random numbers happened to fall a certain way. Your neighbour has different ones. The server has different ones again. The same code then gives a different answer, or no answer at all.

Professionals close that gap with three habits, and you will build all three today.

The reproducibility triangle

A virtual environment is a private folder holding one Python and one set of libraries, used by one project only. Installing something for this course then cannot break another project on the same laptop.

A pinned requirements file says exactly which version of each library you used: `numpy==2.5.1`, not merely `numpy`. The first is a promise a colleague can keep. The second is a wish.

A seed fixes the randomness. Machine learning uses random numbers constantly: which rows go into training, where a model starts. Random means different on every run, which means results nobody can check. A fixed seed makes the randomness repeat, so your result becomes verifiable.

Together these give **reproducibility**: the same code, with the same data, at the same versions, gives the same answer on any machine on any day. Everything else in this course is built on that one property. Experiment tracking is meaningless without it, and a container is only a way of shipping it.

17.3 The walkthrough

The walkthrough works exactly as shipped and cannot be got wrong. Read each step, run it, and look at what it prints before moving on.

Step	What it does	What a correct run shows
0	Checks the workbench	The Python version, then All good.
1	Asks which Python is running	A path containing <code>.venv</code> , and Inside <code>.venv : True</code>
2	Lists what is installed	Four library names with version numbers
3	Freezes those versions into a requirements file	Four lines, each containing a double equals
4	Draws random numbers with no seed	Three numbers that are <i>different</i> every time you run it
5	Draws random numbers with a seed	Two identical rows, and <code>identical : True</code>
6	Builds the dataset twice and compares checksums	Two matching fingerprints, and <code>identical files: True</code>
7	Looks at the data	<code>rows, columns: (600, 5)</code>
8	Records what was run	A small JSON file naming the Python version, the seed and the library versions
9	Saves the work in Git	One line of log output: a short hash followed by the commit message

Two of these deserve your full attention.

Step 4 followed by Step 5 is the point of the whole practical. Step 4 has no seed, so it wanders. Step 5 has a seed, so it always starts from the same place. Both are working correctly. If that feels like a contradiction, you have not yet understood seeds, and the oral assessment will find it.

Step 6 is proof rather than opinion. A checksum is a fixed length fingerprint of a file's exact contents. Change one digit anywhere in the file and the fingerprint changes completely. Two runs producing the same fingerprint is evidence that the two runs produced byte identical data.

17.4 Your tasks

Each task has a stub marked with a TODO comment and a hint. Do them in order.

Task	What to do	Accepted when
T1	The dataset was built with seed 42. Build it with seed 7 and report the first three distances.	Your answer holds three numbers and they match what seed 7 actually produces.
T2	Create <code>work/my_requirements.txt</code> with exactly three lines, one each for <code>numpy</code> , <code>pandas</code> and <code>scikit-learn</code> , each pinned with a double equals to the version installed in your environment.	The file exists and all three libraries are pinned.

Task	What to do	Accepted when
T3	Write a function <code>fingerprint(path)</code> that takes the path of a CSV file and returns a dictionary with the keys <code>rows</code> , <code>sha256</code> and <code>seed</code> .	The dictionary has exactly those three keys, reports 600 rows and seed 42, and the checksum matches the file on disk.

Two traps in Task T2

The library is named `scikit-learn`, not `sklearn`, in a requirements file, even though you import it as `sklearn`. And the versions must be the ones on your own machine, read from your environment, not copied from this handout.

17.5 The self check

The last cell marks your work and prints a table of seven PASS or FAIL lines. It never breaks your notebook, and you may run it as often as you like. A task you have not started simply shows FAIL. A finished student sees seven of seven checks passed.

#	The check	What a FAIL usually means
1	T1 holds three numbers	You left the stub empty, or returned a whole array
2	Those are the first three distances for seed 7	You changed the count or the range as well as the seed
3	The requirements file exists	You wrote it to the wrong folder
4	It pins all three libraries	You wrote <code>sklearn</code> , or left a version off
5	The fingerprint function returns the three required keys	A key is misspelled, or the function returns nothing
6	It counts 600 rows and uses seed 42	You hardcoded a number instead of reading the file
7	The checksum matches the file on disk	You fingerprinted the wrong file

If the self check shows FAIL but your code looks right, restart the kernel and run the notebook from the top. Nine times out of ten, a cell was edited after the check ran.

17.6 What to submit

1. The notebook, with every cell run and its output visible.
2. The requirements file you wrote in Task T2.
3. A screenshot of the output of the last walkthrough cell, the commit log line.

Submission is made at the LMS submission point for your section. If you used any AI assistance, disclose it in the README of your submission, as required by Section 13.2.

17.7 How this practical is marked

The marks are the 4 mark Continuous Lab Evaluation rubric, awarded in the session. The self check table is the evidence for the execution mark.

Self check result	Execution mark
Six or seven of seven passed, walkthrough outputs visible	1.0
Three to five passed	0.5
Fewer than three, or walkthrough outputs not visible	0.0

17.8 Questions you must be ready to answer

The oral assessment carries half the marks for the session. These are the questions this practical is built to test.

1. Which Python is running your notebook, and how do you know it is the right one?
2. What does a virtual environment isolate, and what breaks in this course if you skip it?
3. What is the difference between writing `numpy` and writing `numpy==2.5.1`?
4. Step 4 gives different numbers each run and Step 5 gives the same row twice. Is something broken? Explain.
5. What does a checksum prove that a screenshot of the numbers does not?
6. Why is `scikit-learn` the correct name in a requirements file when you import `sklearn`?
7. Explain add, commit and push in one sentence each.
8. Open your commit on GitHub. Which line changed, and why?
9. Why will this course refuse to put datasets inside Git later on?
10. Your fingerprint reports seed 42. Where did that number come from, and why does the seed belong in the record at all?
11. A classmate cannot reproduce your result. Using only what you built today, what would you send them?
12. What single artifact from today would let someone rebuild your environment in six months?

18 The laboratory roadmap

Thirteen practicals build one running project. Each picks up exactly where the last one stopped.

#	Practical	CO	Extra soft-ware	Primary manual
P01	Your MLOps Workbench	CO1	none	docs.python.org, git-scm.com
P02	Your First Honest Model	CO2	none	scikit-learn.org
P03	Choosing a Model Honestly	CO2	none	scikit-learn.org
P04	From Notebook to Package	CO3	none	docs.python.org

#	Practical	CO	Extra software	Primary manual
P05	Settings in a File, Bugs Caught by a Robot	CO3	none	docs.pytest.org
P06	Remembering Every Experiment	CO5	none	mlflow.org
P07	Putting the Model Behind an Address	CO4	none	fastapi.tiangolo.com
P08	Packing the Service into a Container	CO3	Docker Desktop	docs.docker.com
P09	Describing the Service to a Cluster	CO4	none	kubernetes.io
P10	Logs You Can Actually Search	CO5	none	docs.python.org
P11	A Robot That Checks Your Work	CO4	none	docs.github.com
P12	A Pipeline You Can Run Again	CO4	none	airflow.apache.org
P13	Watching a Model Get Worse	CO5	none	docs.evidentlyai.com

Every practical is CPU only and sized for an 8 GB machine. Peak memory across the semester never exceeds about 900 MB, which occurs in Practical 08.

PART IV

Resources

19 Reading list

19.1 Prescribed textbooks

1. Ameisen, E. (2020). *Building Machine Learning Powered Applications: Going from Idea to Product*. O'Reilly Media.
2. Lauchande, N. (2021). *Machine Learning Engineering with MLflow*. Packt Publishing.

19.2 Reference books

3. Gift, N. and Deza, A. (2023). *Practical MLOps: Operationalizing Machine Learning Models*. O'Reilly Media.
4. Pote, S. (2023). *Machine Learning in Production*. BPB Publications.
5. Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
6. Treveil, M. et al. (2020). *Introducing MLOps: How to Scale Machine Learning in the Enterprise*. O'Reilly Media.

Ask the central library about institutional access to the O'Reilly platform, which covers several of these titles at no personal cost. Low priced Indian editions of the O'Reilly titles are also available.

19.3 Free and openly available

Resource	Where
Sambasivan et al., Data Cascades in High Stakes AI, CHI 2021, open access	https://dl.acm.org/doi/10.1145/3411764.3445518
Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015	https://papers.nips.cc
Treveil et al., Introducing MLOps, sponsored by Dataiku, free after registration	https://www.dataiku.com
Databricks, The Big Book of MLOps, free PDF	https://www.databricks.com/wp-content/uploads/2022/09/The-Big-Book-of-MLOps-v6-082322.pdf
Machine Learning in Production, DeepLearning.AI, free to audit	https://www.coursera.org/learn/introduction-to-machine-learning-in-production
MLOps Zoomcamp, DataTalks.Club, open source	https://github.com/DataTalksClub/mlops-zoomcamp
Stanford CS 329S, Machine Learning Systems Design, open notes	https://stanford-cs329s.github.io

Resource	Where
NPTEL Cloud Computing, IIT Kharagpur	https://nptel.ac.in/courses/106105167

20 Official documentation, the real textbook of this course

The toolchain changes faster than any book. Learning to work from the official manual is an explicit outcome of this course, mapped to PO12.

Topic	Official manual
Python setup and usage	https://docs.python.org/3/using/index.html
Virtual environments	https://docs.python.org/3/library/venv.html
pip requirements file format	https://pip.pypa.io/en/stable/reference/requirements-file-format/
NumPy random generator and seeds	https://numpy.org/doc/stable/reference/random/generator.html
pandas user guide	https://pandas.pydata.org/docs/user_guide/index.html
Pro Git, the complete book, free	https://git-scm.com/book/en/v2
GitHub personal access tokens	https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens
JupyterLab installation	https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html
scikit-learn user guide	https://scikit-learn.org/stable/user_guide.html
DVC, get started	https://doc.dvc.org/start
pytest, get started	https://docs.pytest.org/en/stable/getting-started.html
MLflow tracking quickstart	https://mlflow.org/docs/latest/ml/tracking/quickstart/
FastAPI	https://fastapi.tiangolo.com
Docker, get started	https://docs.docker.com/get-started/
Docker Desktop on Windows	https://docs.docker.com/desktop/setup/install/windows-install/
Kubernetes concepts	https://kubernetes.io/docs/concepts/
GitHub Actions quickstart	https://docs.github.com/en/actions/get-started/quickstart
GitHub Actions workflow syntax	https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions

Topic	Official manual
Apache Airflow quick start	https://airflow.apache.org/docs/apache-airflow/stable/start.html
Airflow, building your first workflow	https://airflow.apache.org/docs/apache-airflow/stable/tutorial/fundamentals.html
Evidently, data and ML checks	https://docs.evidentlyai.com/quickstart_ml
Apache Kafka documentation	https://kafka.apache.org/documentation/
Grafana documentation	https://grafana.com/docs/

Documentation drift, current as of August 2026

Read this before you copy code from any older tutorial or video.

- **Evidently** changed its API. The current documentation imports presets from the presets module. Older material applies to versions 0.6.7 and lower and uses a different module path. Mixing the two produces import errors that look like installation faults.
- **Airflow 3.x** starts with the standalone command, and the web component is now the API server rather than the webserver. Airflow 3.2.0 and later support Python 3.10 through 3.14. On Windows, the practical route is Windows Subsystem for Linux.
- **MLflow** reorganised its documentation paths, and the machine learning material now sits under a dedicated section. Current MLflow versions also expect a backing store, so older tutorials that write to a bare local runs folder will error.
- **DVC** documentation is served from a new host. The older paths still resolve, but the canonical location has moved.
- **GitHub Actions** moved its quickstart page.

21 Frequently asked questions

The best 10 of 15 laboratories count. Can I skip five?

The rule exists for illness and genuine clashes. The advanced laboratories carry the heaviest oral weight and depend on everything built before them, so skipping late is the expensive choice, and skipping early means arriving at those sessions without a working project.

Can I use a different tool for my project?

Not this semester. The oral assessments, the codathons and the end semester paper are all set against the fixed list in Section 7.

Do I need a powerful laptop or a GPU?

No. Every practical is designed for an 8 GB machine with no GPU. If your laptop struggles, the constraint is usually an open browser rather than the hardware.

I have never used Docker, Kubernetes or a cloud platform. Is that a problem?

No. Each is introduced from first principles, and Kubernetes is treated conceptually rather than as laboratory work.

Can I use AI tools to help me?

Yes, as learning aids, with disclosure in your README. You must be able to explain and modify everything you submit. Work you cannot explain in the oral assessment is treated as plagiarism.

What happens if I miss a laboratory?

That session scores zero on all components. Two consecutive absences trigger the parent communication process. There is no separate make up session, which is precisely why the best 10 of 15 rule exists.

Do the Socio-Ed quizzes count towards my marks?

No. They are formative pace checks. They do influence whether a remedial session is scheduled for your section.

I am weak in Python. Should I drop this course?

No. The machine learning bridge sessions and the learner support cohorts exist for exactly this. Tell your faculty in week one rather than in week eight.

PART IV

Appendices

A Before the next session: your checklist

Done	Task
☐	Create a GitHub account, using an address you will keep after graduation
☐	Install Python 3.12 or newer, ticking Add python.exe to PATH on Windows
☐	Install Git, and set your name and email in the global configuration
☐	Create a personal access token on GitHub, with the repo scope
☐	Clone the course repository and create the virtual environment
☐	Install the course libraries and register the notebook kernel
☐	Confirm that Step 0 prints All good and Step 1 reports Inside .venv True
☐	Read Chapter 1 of Practical MLOps, or an equivalent free resource from Part IV
☐	Bring a charged laptop to the laboratory

B Acknowledgement of the course handout

Detach, complete and return to your faculty in the second session.

I confirm that I have received and read the SCSE3040 course handout for Semester V, Odd Semester, Session 2026 to 27. I understand the assessment scheme, the laboratory evaluation rubric, the attendance rule, and the academic integrity and AI usage policy stated in Section 13.2.

Name: _____ Enrolment _____ number: _____

Batch: _____ Signature _____ and _____ date: _____

C Faculty annex

Not for circulation to students. Set the FacultyEdition flag to false in the configuration block to remove this appendix before printing student copies.

C.1 Before the session

1. Run the complete notebook yourself once, on a laboratory grade machine rather than on your own configured laptop. It takes about three minutes when everything works, and the failures you meet are exactly the failures the room will meet.
2. Confirm the repository address, and that the lock file is present in the released package. Practical

01 cannot start without both.

3. Confirm the LMS submission point is live and accepts a work link and a screenshot.
4. Prepare the offline wheel bundle described in Section 15.9. If the campus link is slow, it is the difference between a completed session and a lost one.
5. Check that Git is present on the laboratory machines. If it is absent, the final step will error but nothing else in the practical breaks.
6. Have the answer to why 42 ready. It comes from The Hitchhiker's Guide to the Galaxy. Students always ask.

C.2 Timing for a 120 minute block

Minutes	Activity
0 to 15	Setup. Everyone opens the notebook and runs Step 0
15 to 30	Steps 1 to 3: which Python, what is installed, freeze the versions
30 to 50	Steps 4 to 6: randomness, seeds, matching checksums
50 to 70	Steps 7 to 9: the dataset, the run record, the commit
70 to 105	Tasks T1 to T3. Conduct the oral assessment during this window, at each student's machine, about ninety seconds each
105 to 120	Self check, discussion of the FAIL lines, submission

If a diagnostic or any other departmental activity must also fit, place it at the end where it needs no machine, or at the very start on paper if the machines are still being prepared.

C.3 Say these things out loud

- Ask the room who has installed something and broken something else. Almost every hand goes up. That is precisely what a virtual environment prevents.
- Run Step 4 twice in front of them. The numbers changing is the moment the idea of a seed lands.
- Dwell on Step 6. A matching checksum is proof, not opinion. Change one digit of the seed and show the fingerprint change completely.
- State plainly that 40 percent of the course marks are earned live, in this room, week by week.

C.4 Where students get stuck, and what to say

They do this	Say this
Conclude the computer is broken when Step 5 repeats a row	Step 4 has no seed so it wanders, Step 5 has a seed so it always starts from the same place. Both are correct
Write an unpinned library name in Task T2	That says any version will do. In six months, any version will be a version that breaks your code
Skip cells, then hit a NameError	A NameError means you have not made that thing yet. Go back up and run the cells in order
Return the wrong file in Task T3	Read the check line in the self check output. It names exactly which key it could not find

C.5 Records

Enter the 4 mark rubric score for every student in your named sheet in the shared Continuous Lab Assessment tracker, and add the Timeline Log row for the session, within seven days of the session.

This handout follows the approved Teaching Learning and Assessment Plan for SCSE3040. Where any detail here differs from the approved plan or from a subsequent circular of the School, the approved plan prevails.